

RESEARCH ARTICLE

Toward an Efficient and Dynamic Allocation of Radio Access Network Slicing Resources for 5G Era

XIAOLEI CHANG^{1,2}, TIAN JI^{1,2}, RUNSU ZHU³, ZHENZHOU WU², CHENXI LI²,
AND YONG JIANG^{2,4}

¹Tsinghua University, Beijing 100000, China

²Research Institute of Tsinghua University in Shenzhen, Shenzhen 518000, China

³Longhua District Government Service Data Administration, Shenzhen 518000, China

⁴Tsinghua Shenzhen International Graduate School, Shenzhen 518000, China

Corresponding author: Zhenzhou Wu (wuzhenzhou@qzcloud.com)

This work was supported in part by the Shenzhen Science and Technology Program under Grant SGDX20190917160803729.

ABSTRACT With the development of 5G technology and Internet of Things (IoT), more and more devices are connected through 5G wirelessly. Radio access network (RAN) slicing, as a key feature of 5G, enables a flexible bandwidth resource allocation policy, and facilitates various types of services to operate on different network slices. However, RAN slicing resources is scarce, thus effective management of wireless bandwidth resources in RAN slicing becomes indispensable to improve user satisfaction. Extensive research has investigated into RAN slicing, but they do not take user mobility into consideration. While RAN slicing allocation has a great impact on user experience in mobile 5G scenarios, user mobility poses great challenges to network management and causing unsatisfaction of users. In this paper, we propose a new RAN slicing allocation strategy based on machine learning, to maximize spectrum efficiency while guaranteeing the Service Satisfaction Ratio (SSR) of various slicing services. To further alleviate the SSR fluctuation brought by user mobility, we study into the temporal characteristics of user mobility and preprocess the state sequences using Long Short-Term Memory (LSTM) networks. Finally, these sequences are taken as the input of an Advantage Actor Critic (A2C) reinforcement learning network to develop a RAN slicing allocation policy. We conduct comprehensive simulations, and the results show that the performance of the proposed mechanism outperforms the traditional mechanism in ensuring SSR and enhancing the spectrum efficiency.

INDEX TERMS Internet of Things, network slicing, reinforcement learning, radio access networks.

I. INTRODUCTION

The development of IoT has led to more and more IoT user devices connecting to the network, each with different service quality requirements. Traditional 4G network provides bandwidth resources to users uniformly, which is not efficient and dynamic. The current 5G network brings new opportunities with many new proposed technologies, such as Mobile Edge Computing (MEC), and RAN slicing technology in the latest 3GPP standard [1]. Among them, RAN slicing technology provides room for future innovation by addressing various

types of user demands through different network slices, which can be seen as virtualized bandwidth resources based on the same physical connection. Network slices can ensure transmission rate, latency, and security, and can be customized according to user requirements, which is much more flexible than previous wireless communication technology.

However, RAN slicing resource is still very expensive and needs to be efficiently orchestrated. However, due to the various types of user requirements, it is difficult to efficiently allocate slices while satisfying user demands. Although many previous works studied slice resource allocations [2], more and more mobile devices appear in 5G scenarios and they pose greater challenges for slicing allocation

The associate editor coordinating the review of this manuscript and approving it for publication was Rentao Gu.

because user experience will fluctuate in mobile states. Thus, an efficient and dynamic allocation mechanism for RAN slicing resources is badly needed in the 5G era.

In this paper, we take user mobility into consideration while allocating 5G slice resources to multiple users from a wireless base station. The system needs to meet two different goals, 1) to maximize the system utility, which indicates the overall SSR, and 2) to predicate the future movement of mobile users and be adaptive when allocating slices to them. To achieve the best of two worlds, we first formulate the problem as an optimization problem that maximizes overall user utility while satisfying bandwidth and mobility constraints. Then we utilize an A2C reinforcement learning-based algorithm, which combines the advantages of value-based and policy-based approaches to train the agent. Through training the decision model in a simulation environment, this approach can efficiently allocate wireless network bandwidth when environment and user demand change.

Although A2C is adaptive to state changes, user mobility patterns are difficult to learn. To further improve the performance of the proposed system, we record the final state sequence under user mobility and then apply an LSTM network to extract the temporal characteristics of these sequences and predicate the future states.

We conduct comprehensive simulations, and the results show that the A2C algorithm model integrated with the LSTM network can accurately allocate wireless bandwidth resources after efficient training, while ensuring the SSR of different users.

The paper is structured as follows: Section II introduces relevant research works, including 5G network slicing and reinforcement learning for resource allocation. Section III describes the framework of our RAN slicing allocation system. Section IV formulates the optimization problem for the wireless network slicing allocation model. Section V proposes the algorithm design, integrating the LSTM network and A2C reinforcement learning. Section VI simulates various scenarios to validate the performance of the proposed algorithm. Finally, Section VII summarized the paper and future work.

II. RELATED WORK

A. REINFORCEMENT LEARNING

Reinforcement learning (RL) is one of the paradigms of machine learning. Data plays a crucial role in the final performance results of machine learning models. Unlike supervised and unsupervised learning, reinforcement learning depends mainly on interacting with the environment to generate data for learning and continually optimizing the model through trial and error. Specifically, in reinforcement learning, an agent generates an action based on the observed environmental state and policy, which affects the environmental state, and receives rewards for optimizing the policy model through this interactive process. At the k th step, the agent observes the environmental state s_k and returns an action a_k to the environment, causing the environmental state

to transition to s_{k+1} , and the agent receives a reward r_k for this action. Since the decision for taking an action depends only on the current state, this process can be viewed as a Markov decision process (MDP). $M = \langle S, A, P_r, R \rangle$, where S represents the set of states, A represents the set of actions, $P_r(s'|s, a)$ represents the probability of taking action a to transition from state s to state s' , and $R(s, a)$ represents the reward for taking action a in state s . Because of the Markov property of the reinforcement learning process, it naturally applies to decision-making tasks based on environmental states, such as industrial control, scheduling decisions, and game AI.

Reinforcement learning can be divided into two categories based on the environmental model: model-based and model-free. Model-based reinforcement learning can obtain the state transition probability of the environmental model to plan decision-making, i.e., the environmental model is known. Model-free reinforcement learning does not predict the state, but constantly optimizes the model through environmental reward feedback. Currently, most of the research on reinforcement learning focuses on model-free reinforcement learning, which includes two types: value-based and policy-based reinforcement learning algorithms. Value-based reinforcement learning is mainly represented by Q-learning, DQN [3], Dueling DQN [4] and Double DQN [5] algorithms. Compared with value-based algorithms, policy-based algorithms can handle discrete/continuous space problems and have better convergence. Moreover, policy-based algorithms tend to fall into the dilemma of local optima due to their large trajectory variance and low sample utilization [6]. The reinforcement learning algorithms used in this paper are mainly policy-based reinforcement learning algorithms, such as REINFORCE, A2C [7], DDPG [8], PPO [9], and SAC [10]. The main feature of policy gradient methods is to directly model and optimize policies. The policy is usually defined as a function $\pi_\theta(a|s)$ with parameter θ . Since the objective function's value is directly affected by the policy, many reinforcement learning algorithms can be employed to optimize θ to maximize the objective function.

B. NETWORK SLICING

With the growing maturity of 5G networks, a variety of terminal devices are accessing wireless networks through 5G technology. These devices generate varying user demands for network services with regard to bandwidth, latency, and reliability. The International Telecommunication Union has identified three distinct categories of 5G application requirements: Enhanced Mobile Broadband (eMBB), Massive Machine-Type Communications (mMTC), and Ultra-Reliable Low-Latency Communications (URLLC). The diverse task requirements and a growing number of device connections raise new challenges to the existing network architectures and resource management.

Network slicing integrates Software-Defined Networking (SDN) and Network Function Virtualization (NFV) technologies to offer independent network resources to different users

with diverse requirements in a flexible manner. As a key technology of 5G, network slicing meets a plethora of user service demands. Specifically, operators create multiple logically independent networks on a shared physical network infrastructure, which forms a logical isolation of network resources among different network slices, ensuring that services on different network slices do not mutually interfere. Besides, in response to varying network resource demands of different slices, network slicing technology can dynamically allocate resources flexibly to different slices on the same physical network. To realize end-to-end network slicing services, network slicing can be divided into core network slicing and RAN slicing. The core network is mainly composed of network servers and their links, abstracting the underlying physical network resources (link bandwidth, CPU resources, storage resources) with SDN and NFV technologies and providing different network services with varying quality of service to upper-layer applications. RAN slicing, on the other hand, primarily allocates network bandwidth to users with different demands, considering the limited network bandwidth of base stations. Balancing maximum bandwidth efficiency and user service quality of different slices within the limited wireless network communication bandwidth is a major research focus of RAN slicing. The research on network slicing in this paper is also focused on the aspect of RAN slicing.

C. RESEARCH STATUS OF NETWORK SLICING IN MEC

As a critical technology for 5G, network slicing plays a vital role in meeting a variety of service demands. While ensuring the isolation of services, dynamically allocating RAN bandwidth resources to ensure service satisfaction poses challenges. Khamse-Ashari et al. [11] conducted resource allocation within a particular network slice to maximize user SSRs in it. Based on a distributed mechanism, the study performed an iterative auction in the network slice and offered price acceptance to service providers to solve the problem. Lieto et al. [12] asked users to make automated decisions based on their own needs and adopted a slice-aware scheduler to achieve Nash equilibrium in decision-making. Some studies used deep neural networks to predict unknown user input traffic and allocate it to appropriate network slices [13]. Some other studies adopted blockchain technology to enhance security in the process of network slicing [14], [15], [16].

Existing research on RAN network slicing mainly focuses on network resource sharing, resource virtualization, slice isolation, mobility management, resource efficiency, security and privacy, dynamic slice creation and management [17], where reinforcement learning-based methods are widely used to seek solutions. Setayesh et al. [18] adopted layered reinforcement learning to improve the throughput and Service Level Agreement (SLA) SSR of eMBB and URLLC service slices. In the research, deep reinforcement learning algorithms were used to adjust slice configuration parameters for long time slots, and a deep neural network based on an attention mechanism was used to allocate wireless resources

to users in eMBB and URLLC service slices for short time slots. Wang et al. [19] jointly optimized the communication, computing, and caching resource allocation in MEC. The optimization objective of this research was to maximize a utility function while ensuring user service quality. A twin-actor Deep Deterministic Policy Gradient (twin-actor DDPG) algorithm was proposed to solve the problem. According to the characteristics of 5G New Radio (NR), Boutiba et al. [20] used deep reinforcement learning to allocate network slice bandwidth to maximize throughput and ensure service quality satisfaction, which verified the effectiveness of their algorithms in larger bandwidth and more user scenarios. Boutiba et al. [21] proposed a flexible slicing resource allocation framework for 5G NR. For industrial IoT scenarios, Mai et al. [22] adjusted the transmission power and spreading factor to satisfy service quality. Messaoud et al. [23] examined the computing resources, service quality satisfaction, and privacy data of industrial IoT and used federated reinforcement learning to adjust the transmission power and spreading factor. For discrete channel assignments and continuous energy harvesting time division, Xu et al. [24] used a discrete-hybrid Soft Actor-Critic (SAC) algorithm to allocate resources. To optimize the SSR and SE of RAN slicing services, Li et al. utilized various reinforcement learning algorithms, such as Deep Q-Network (DQN) [25], multi-agent reinforcement learning based on graph attention network [26], and distributed DQN algorithm based on the generative adversarial network [27]. Some other studies used multi-agent reinforcement learning methods for parallel training to speed up the model training process [28], [29]. Regarding the time series problem caused by the mobile characteristics of users in MEC, Cui et al. [30] and Li et al. [31] combined LSTM networks with reinforcement learning to allocate network-slicing resources to ensure the service SSR among mobile users. Liu et al. [32] combined the alternating direction multiplier method with deep reinforcement learning to address it.

The allocation of network-slicing resources requires adaptive and dynamic resource allocation decisions, and the features of MDP in reinforcement learning algorithms suggest that they are naturally suited for solving such problems. Therefore, the aforementioned research proposed a large number of reinforcement learning algorithms for network slicing resource allocation based on the characteristics of different reinforcement learning algorithms to achieve high performance. This study also adopts a reinforcement learning-based approach to address the dynamic allocation of bandwidth resources in RAN, with a view to ensuring the service SSR of multiple network slicing and improving the SE of wireless bandwidth.

III. SYSTEM SCENARIO

In order to provide users with end-to-end network slicing, it is necessary to simultaneously implement core network slicing and RAN slicing. With the rapid increase in 5G network coverage and the exponential growth of IoT devices, more

user devices are accessing the network via wireless means. Therefore, the intelligent network slicing system scenario in this paper mainly considers the deployment of RANs. Fig. 1 shows a wireless network resource dynamic allocation framework based on SDN architecture in the RAN scenario. The system scenario mainly includes the following elements:

- 1) User: Through the RAN bandwidth signal of the base station, users generate different applications according to their own needs. Therefore, wireless networks should provide network resources of different quality levels. At the same time, users move at a certain rate within the range of wireless signals of the base station, and different users have different requirements for data packets based on the distribution of their spatial locations.
- 2) Wireless base station: Providing RAN services to users within a certain range, offering data transmission services with limited wireless bandwidth resources (e.g. 10 MHz).
- 3) Network slicing: Abstracting wireless bandwidth resources to divide them into logically independent networks, with slices not affecting each other. According to the service type, different slices provide network resource services of different levels, i.e. different allocated resources. Based on changes in slice service requirements, the network resource allocation in time series is adjusted to meet different service needs.
- 4) Service: Classifying the services provided by slices based on the network service requirements of the user's application type, such as VoLTE providing call services, eMBB providing online video, VR, AR and other services, URLLC providing autonomous driving, remote medical and other services.

Central controller: Maintaining user state information, base station wireless network resource information, service information, etc. as the system's global information collector. Serving as the controller in the wireless network slicing scenario, sending control signals based on the allocation decisions of wireless network resources, and managing and scheduling wireless resources. The intelligent network slicing algorithm in this section is deployed in the central controller to realize the bandwidth resource decision-making function of network slicing.

This section mainly adopts the frequency division duplexing (FDD) scheme to allocate network bandwidth for the downlink, where the transmission of the uplink and downlink are carried out at different frequencies. Besides, it is assumed that a series of network slices $1, 2, \dots, N$ share limited wireless bandwidth W , with each slice providing network resources for a certain type of user $\mathcal{U}_n$. The changing demand of each slice is represented by vector $\mathbf{d} = [d_1, d_2, \dots, d_N]$. Based on the quantity of demand, service type, and decision algorithms of different slices, the system allocates different bandwidth resources $= \{w_1, w_2, \dots, w_n\}$ to them, with the bandwidth allocated to slices represented by w_n . The

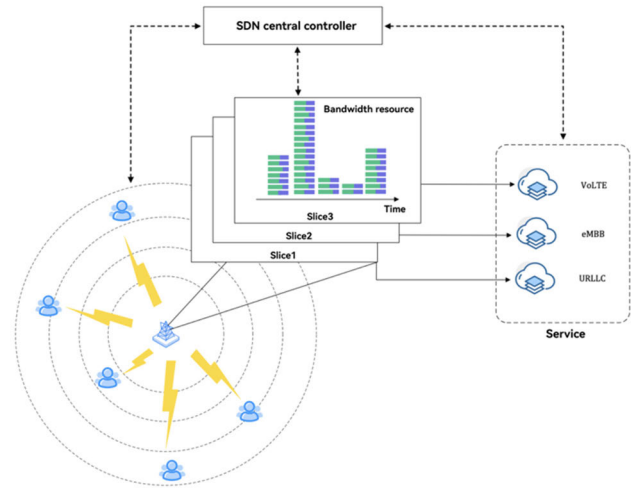


FIGURE 1. System scenario of network slicing.

demand $\mathbf{d}$ of slices varies according to the distribution of user mobility.

IV. PROBLEM MODEL

The system aims to maximize system utility, which is a weighted combination of SE and SLA SSR. First, the Shannon theorem can be employed to calculate the downlink rate r_{u_n} of user u_n in slice n :

$$r_{u_n} = w_n \log(1 + SNR_{u_n}), \quad \forall u_n \in \mathcal{U}_n \quad (1)$$

where, SNR_{u_n} is the signal-to-noise ratio of the user u_n with the base station. Through the downlink rate of the user, SE can be calculated using the following formula:

$$SE = \frac{\sum_{n \in \mathcal{N}} \sum_{u_n \in \mathcal{U}_n} r_{u_n}}{W} \quad (2)$$

The SSR of slice n can be obtained by calculating the total number of successfully transmitted data packets and the total number of received data packets. In this section, P_{u_n} is defined as the data packet transmitted by the base station for user u_n , and $a_{p_{u_n}}$ is used to indicate whether a specific data packet p_{u_n} is successfully transmitted. Only when the user's transmission rate is greater than the required rate for the service level $\bar{r}_n$, and the response latency of the data packet is less than the maximum latency that the service level can bear $\bar{l}_n$, that is, $r_{u_n} \geq \bar{r}_n$ and $l_{p_{u_n}} \leq \bar{l}_n$, $a_{p_{u_n}} = 1$. Therefore, SSR can be calculated using the following formula:

$$SSR = \frac{\sum_{u_n \in \mathcal{U}_n} \sum_{p_{u_n} \in P_{u_n}} a_{p_{u_n}}}{\sum_{u_n \in \mathcal{U}_n} P_{u_n}} \quad (3)$$

So the problem can be described as:

$$\max_{\mathbf{w}} \alpha \cdot SE(\mathbf{d}, \mathbf{w}) + \beta \cdot SSR(\mathbf{d}, \mathbf{w}) \quad (4)$$

And it follows $\sum_{n \in \mathcal{N}} w_n = W$, $\sum_{u_n \in \mathcal{U}_n} P_{u_n} = d_n$, $w_i = k \cdot \omega$, $\forall i \in [1, 2, \dots, N]$. Where, α and β are weight factors for SE and SSR, respectively, and $\beta = [\beta_1, \beta_2, \dots, \beta_N]$ represents the weight factors for each slice. ω is the minimum unit of bandwidth allocation.

During the course of this research, several technical challenges were encountered. On one hand, users exhibit a multitude of characteristic attributes, making feature selection somewhat intricate. We employed LSTM to capture the temporal features of user mobility, facilitating the development of slice allocation strategies. On the other hand, achieving realistic simulation of slice application scenarios posed certain challenges. We established a model for wireless network slice applications and devised various simulation scenarios encompassing extensive user populations, rapid connections, high bandwidth, and mixed-use environments.

V. REINFORCEMENT LEARNING-BASED RESOURCE ALLOCATION ALGORITHM FOR NETWORK SLICING

A. A2C ALGORITHM

Similar to other reinforcement learning methods, the Advantage Actor-Critic (A2C) algorithm [7] used in this section also continuously optimizes the model through interactive trial and error with the environment, which can be viewed as an MDP. At the core of A2C lies an actor-critic framework, where the actor is responsible for decision-making, determining which action (i.e., resource allocation strategy) to take given a state, while the critic estimates the value of this decision. In the context of resource slicing, the actor can be represented as a neural network that takes environmental states as input and outputs a probability distribution representing the allocation probabilities for different slices. The critic can be another neural network used to estimate the value function of the current state. At each time step, an allocation action for resource slicing is sampled based on the actor's output probability distribution. Subsequently, using environmental feedback including reward signals and the next state, the advantage is computed, reflecting the superiority of the current action relative to the average policy. With the advantage, policy gradients for the actor and value function gradients for the critic are calculated. The neural network parameters of both the actor and critic are updated using these computed gradients. This gradual update of parameters causes the actor's policy to gradually adjust towards better slice allocation strategies, and the critic's value function estimation to progressively converge to more accurate state values. By iteratively executing the aforementioned steps, continuously interacting with the environment and updating actor and critic parameters, the resource allocation strategy is optimized.

Reinforcement learning algorithms aim to achieve maximum cumulative expected reward $R = \sum_{k=0}^{\infty} \gamma^k r_k$. The action value at step t can be evaluated through $Q(s_t, a_t) = \mathbb{E}[R_t | s_t = s, a_t = a]$, while the state value $V(s_t) = \mathbb{E}[R_t | s_t = s]$ can only be estimated based on the current transitional state. A2C only adopts the state value function V for training, thereby reducing the number of parameters and simplifying the training process. The advantage in A2C can be obtained through

$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, which can usually be computed with minimal bias using the TD error:

$$A(s_t, a_t) \approx r_t + \gamma V(s_{t+1} | s_t, a_t) - V(s_t) = \delta(s_t) \quad (5)$$

The gradient of the actor network is $\nabla_{\theta} \log \pi(a_t | s_t; \theta) \delta(s_t)$, and the loss function of the critic network is $\mathcal{L}_{critic} = \delta(s_t)^2$.

B. LSTM

Due to the mobility of users within the wireless access range, and the position distribution resulting from user mobility affecting the amount of transmitted data packets P_{u_n} , the data packet demand of each slice d undergoes dynamic changes as users move. Recurrent neural networks (RNNs) are a type of neural network used to process serial data. Compared with general neural networks, they are more capable of handling serial data with changes. To capture the moving features of users in the time series, this section uses an LSTM network [33] to preprocess the environmental state. LSTM effectively captures temporal features of user mobility states through internal gating mechanisms and memory units. It selectively retains and forgets information, accommodating varying mobile patterns across different time steps, managing intricate temporal sequence dependencies, and preserving both long-term and short-term mobility patterns. Specifically, a queue is used to store the observed state of the environment at each step in the time series, the past two observed sequences are extracted and preprocessed by an LSTM network, and then the preprocessed states of the two observed sequences are concatenated as inputs to the actor and critic networks. By preprocessing past observed states through an LSTM network, historical state information is encoded into cell states, facilitating a more accurate capture of user mobility patterns and trends. These temporal features' influence is then taken into account during the decision-making process.

As a variation of RNNs, the core concept of LSTM lies in its cell state and "gate" structure. The cell state serves as a path for information transmission, allowing information to be propagated throughout the sequence, and can be viewed as the "memory" of the network. In theory, the cell state can carry relevant information throughout the sequence processing, so even earlier time steps can carry information to later time steps in the cell, overcoming the influence of short-term memory. The addition and removal of information can be achieved through the "gate" structure, which learns which information to keep or forget during the training process. Fig. 2 shows the structure of LSTM. Where, c_{t-1} represents the previous LSTM cell state, h_{t-1} represents the previous LSTM cell hidden state, $\otimes$ represents the product of positions of matrices of the same size, $\oplus$ represents matrix addition and σ represents activation function *sigmoid*. There are three gates in LSTM: the forget gate, the input gate, and the output gate. The forget gate selectively forgets the input from the previous stage, that is, it chooses to remember important information and forget unimportant information. The input gate updates the current cell state, deciding which

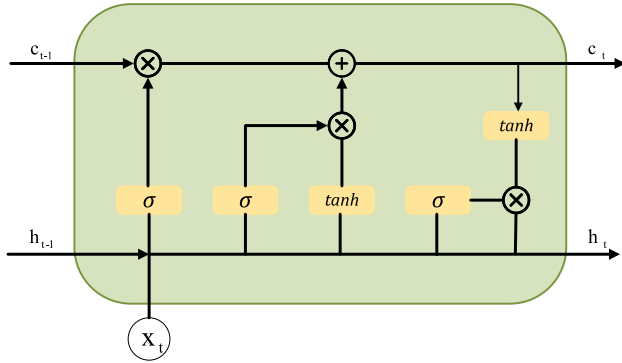


FIGURE 2. Structure of LSTM.

information needs to be updated. The output gate determines the next hidden state output based on the current input and the previous hidden state.

In LSTM, c_t represents the current cell state, h_t represents the current hidden state, and x_t represents the current input state. The three gate states are defined as z_i , z_f , and z_o . They are numbers between 0 and 1 obtained by concatenating x_t and h_{t-1} , multiplying by weight matrix w , adding bias factor b , and outputting through the *sigmoid* function. The three states can be represented by the following formulas:

$$z_i = \sigma(w_i[x_t, h_{t-1}] + b_i) \quad (6)$$

$$z_f = \sigma(w_f[x_t, h_{t-1}] + b_f) \quad (7)$$

$$z_o = \sigma(w_o[x_t, h_{t-1}] + b_o) \quad (8)$$

Based on the above gate states, c_t , h_t can be obtained by the following formulas:

$$c_t = z_f \otimes c_{t-1} + z_i \otimes \tanh(w_c[x_t, h_{t-1}] + b_c) \quad (9)$$

$$h_t = z_o \otimes \tanh(c_t) \quad (10)$$

The final output y_t can frequently be obtained through $y_t = \sigma(w_y h_t + b_y)$

C. ALGORITHM

Based on the introduction of the A2C algorithm and the role and structure of LSTM described above, the reinforcement learning algorithm framework adopted in this section is shown in Fig. 3. The structure of the algorithm consists of three parts:

- 1) State sequence preprocessing network: This network mainly captures the temporal features of various service request state sequences, including two queues that hold the states, a fully connected layer, and an LSTM layer. Specifically, the two queues store two state sequences of different lengths in chronological order, enabling the LSTM network to process state features at different time intervals. The two state sequences are input to the fully connected layer to extract features, and then serve as input to the LSTM layer, which updates the cell state and hidden state through L_1 and L_2 iterations. The final hidden states h_t of the two sequences can be concatenated as a feature s'_t that

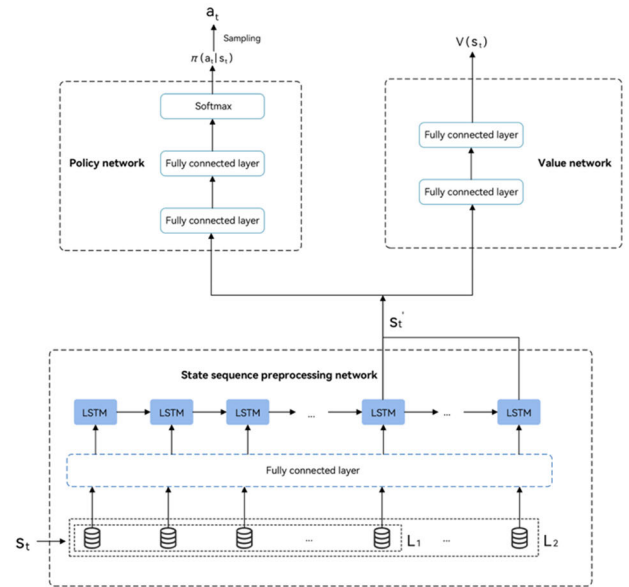


FIGURE 3. Framework of A2C algorithm with LSTM preprocessing network.

represents past L_1 and L_2 states, and provided input to the policy network and value network.

- 2) Policy network: This network mainly updates decision policies and generates actions. The preprocessed state feature s'_t is used as input and passed through two fully connected layers. The size of the neurons in the second fully connected layer is the size of the action space $|A|$. Then, the action probability distribution, or policy $\pi(a_t | s_t)$, is obtained through the softmax function. Finally, action a_t is selected by sampling and executed in the environment.

Value Network: This network mainly evaluates the value of the current state. The preprocessed state feature s'_t is used as input and is outputted through two fully connected layers to obtain the value $V(s_t)$ of the current state. In the A2C algorithm, the value network also needs to calculate the value $V(s_{t+1} | s_t, a_t)$ of the next state based on the changed environmental state after the action is executed, in order to calculate the TD error.

In the actor network, an entropy term with a factor β of 0.01 is added to encourage the policy network to explore and select other actions in the action space for better policies. According to the gradient of the actor network described in section V, the loss function for the actor network can be deduced as follows:

$$\mathcal{L}_{actor} = -\delta_t(s_t; \theta_c) \log \pi(a_t | s_t; \theta_a) - \beta H(\pi(a_t | s_t; \theta_a)) \quad (11)$$

The parameter update of the actor network can be described as:

$$\theta_a \leftarrow \theta_a + \delta_t(s_t; \theta_c) \nabla_{\theta_a} \log \pi(a_t | s_t; \theta_a) + \beta \nabla_{\theta_a} H(\pi(a_t | s_t; \theta_a)) \quad (12)$$

The critic network adopts the squared TD error as the loss function, expressed by the following formula:

$$\mathcal{L}_{critic} = \delta_t^2 = (r_t + \gamma V(s_{t+1}; \theta_c) - V(s_t; \theta_c))^2 \quad (13)$$

The parameter update of the critic network can be described as:

$$\theta_c \leftarrow \theta_c + \delta_t(s_t; \theta_c) \nabla_{\theta_c}(s_t; \theta_c) \quad (14)$$

Considering the aim of maximizing system utility, the system is composed of a weighted combination of SE and SSR from each slice. To prevent policies in the algorithm from sacrificing the SSR of certain slices in order to maximize SE, the reward during training is set to be segmented. Only when the SSR of all slices exceeds the set threshold will additional rewards for SE optimization be reflected in the reward function. In this section's training, three network slices are used to provide corresponding resources for VoLTE, eMBB, and URLLC services. q_v , q_e , and q_u are defined as the SSR of the VoLTE, eMBB, and URLLC service slices, respectively. The reward function can be represented by the following formula:

Algorithm 1 A2C Training Algorithm With LSTM

```

01   Initializing parameters  $\theta_a$  and  $\theta_c$  of actor and critic
    networks
02   Initializing two queues  $Q_1$  and  $Q_2$  of lengths  $L_1$ 
    and  $L_2$ 
03   for 1 to  $L_2$  do
04       Randomly sampling action  $a_{rand}$  in action space
     $\mathcal{A}$  and feeding it back to the environment
05       Observing the environment to obtain  $obs_t$  and
    inserting it into  $Q_2$ 
06       if  $i < L_1$  then  $obs_t$  and inserting it into  $Q_1$ 
07   end for
08   for  $t = 1$  to the maximum number of iterations do
09       Processing the states in queues  $Q_1$  and  $Q_2$  by
    LSTM network and concatenating them to obtain  $s'_t$ 
10       Inputting  $s'_t$  into the actor network to obtain the
    action probability distribution  $\pi(a_t|s_t)$ 
11       Inputting  $s'_t$  into the actor network to obtain the
    state value  $V(s_t)$ 
12       Sampling to obtain action  $a_t$  according to action
    probability distribution  $\pi(a_t|s_t)$  and executing it in
    the environment
13       Environment feedback reward  $r_t$  and
    newenvironmental state  $obs_t$ 
14       Removing the first elements in queues  $Q_1$  and
     $Q_2$ , and inserting  $obs_t$  into  $Q_1$  and  $Q_2$ 
15       Processing the states in queues  $Q_1$  and  $Q_2$  by
    LSTM network and concatenating them to obtain  $s'_{t+1}$ 
16       Inputting  $s'_{t+1}$  into the critic network to obtain the
    state value  $V(s_{t+1})$ 
17       Calculating the TD error  $\delta_t(s_t) = r_t + \gamma V(s_{t+1}) - V(s_t)$ 
18        $\theta_c \leftarrow \theta_c + \delta_t(s_t; \theta_c) \nabla_{\theta_c}(s_t; \theta_c)$ 
19        $\theta_a \leftarrow \theta_a + \delta_t(s_t; \theta_c) \nabla_{\theta_a} \log \pi(a_t|s_t; \theta_a) + \beta \nabla_{\theta_a} H(\pi(a_t|s_t; \theta_a))$ 
20        $t \leftarrow t + 1$ 
21   end for

```

According to the formula, the reward function is divided into four segments. When the SSR of all slices meets the set threshold and SE is greater than or equal to 280, positive feedback rewards greater than 4 will be given based on the degree of SE optimization. When SE is less than 280 but

the SSR of all slices meets the threshold, moderate positive feedback rewards will be given. When the SSR of VoLTE and eMBB services meets the threshold but the SSR of the URLLC service does not meet the threshold, the reward will be calculated based on the SSR of URLLC. If $q_u > 0.7$, positive feedback rewards will still be given, but when q_u is small, negative feedback rewards even smaller than -5 will be given. This is because the URLLC service has high requirements for latency and relatively large data packets, making it difficult to guarantee its SSR in limited wireless network resources. Thus, when $q_u > 0.7$, positive feedback rewards will still be given, and when $q_u = 0$, the maximum negative feedback penalty will be given. Since URLLC tasks are usually critical tasks that require high quality of service and high priority (e.g. remote medical, autonomous driving, and remote industrial control), their SSR should be ensured as much as possible. When the SSR of either VoLTE or eMBB service is below the threshold, relatively low negative feedback penalties will be given directly.

As described in the previous section, in order to quickly map the environmental state to bandwidth allocation decisions, this paper adopts a reinforcement learning algorithm A2C to train the agent. Also, to achieve better performance when there are fluctuations in the data packet demand vector d generated by user mobility, the algorithm in this section incorporates an LSTM network to preprocess the observed environmental state to obtain temporal information. The training process of the algorithm is shown in The training process of the algorithm is shown in Algorithm 1. First, in lines 1-2 of the algorithm, parameters θ_a and θ_c of the actor and critic networks are initialized, and two queues are set up with different lengths ($L_1 = 10$, $L_2 = 100$) to store the observed environmental state. Next, L_2 actions are randomly sampled and executed in the environment to obtain L_2 environmental states stored in Q_2 , and L_1 environmental states stored in Q_1 . These states are stored in two queues according to their temporal sequence (lines 3-7). Lines 8-21 represent the training process of the algorithm. The states in the two queues are first preprocessed by the LSTM network, which are concatenated to obtain the temporal sequence state s'_t of the past L_1 and L_2 environmental states. The state is then used as input to the actor and critic networks to obtain the action probability distribution $\pi(a_t|s_t)$ and the state value $V(s_t)$. The action is then sampled from the action probability distribution and executed in the environment. The environmental state changes after implementing the action, returning to the environmental state obs_t and the reward r_t at time step t . The new environmental state is inserted into queues Q_1 and Q_2 , and the first elements in the queues (the environmental state farthest from step t) are removed. The states in the two queues are concatenated and processed through the LSTM network to obtain the temporal state s'_{t+1} , and $V(s_{t+1})$ is calculated through the critic network. Using the formula for TD error, $\delta_t(s_t)$ can be calculated, and parameters θ_a and θ_c of actor and critic networks can be updated according to the TD error. Finally, it moves on to the next

TABLE 1. Training parameters.

Training parameters	
Learning rate of actor network	$2 * 10^{-3}$
Learning rate of critic network	$1 * 10^{-2}$
Weight of entropy term	$1 * 10^{-3}$
Maximum number of iteration rounds	10,000
Number of state features	3
Action space	[1128,3]
Size of LSTM cells	64
Size of fully connected layer for actor	[32,1]
Size of fully connected layer for actor critic	[32,1]
Size of statequeues Q_1 and Q_2	10,100

round of training until the maximum number of iterations is reached.

Table 1 lists the parameters trained by Algo. 1, with a small entropy weight set to encourage the agent to explore the action space. A larger learning rate for the critic network is used to learn the value function, which guides actor optimization through advantage calculation. The algorithm is set to default to 10,000 training iterations. The number of input features for the network is 3, representing the number of service packets over 3 slices. The action space size is [1128,3], with 1128 being the maximum assignable bandwidth divided by the minimum bandwidth allocation unit, which makes the decision for the number of allocated bandwidths of the three slices. The input state is first preprocessed by an LSTM network of size 64, then passed through two fully connected layers in both the actor and the critic networks. The actor network finally obtains the probability distribution of actions through a *softmax* function. Two queues of different lengths are used to store environment states for LSTM preprocessing, including the last 10 and 100 rounds. If the number of slices and total bandwidth set by the environment change, the corresponding parameters should be adjusted accordingly. The simulation experiment uses the parameter training model in Table 1 by default.

VI. SIMULATION AND PERFORMANCE ANALYSIS

A. SIMULATION ENVIRONMENT SETTINGS

This section mainly focuses on resource allocation for RAN network slicing. In the simulation experiment, this paper considers three typical service types in wireless networks: VoLTE, which provides wireless network call services; eMBB, which provides high bandwidth network services, such as mobile entertainment and augmented reality; and URLLCS, which provides low latency and high-reliability network services, such as remote control and autonomous

TABLE 2. Parameters of simulation environment.

Type of slicing service	VoLTE	eMBB	URLLC
Bandwidth		10 MHz	
Minimum bandwidth allocation unit		200 kHz	
Bandwidth adjustment interval		1,000 ms	
Number of user devices	200	400	600
User movement speed	1 m/s	4 m/s	8 m/s
User reach interval distribution	Uniform distribution (min: 0ms, max: 160ms)	Pareto distribution (exponential parameter: 1.2, mean: 6ms, max: 12.5ms)	Exponential distribution (mean: 180ms)
Packet size distribution	Constant (40 bytes)	Pareto distribution (exponential parameter: 1.2, mean: 100 words/nodes, max: 250 bytes)	Constant (0.3 megabytes)
SLA			
Rate	51 kbps	100 Mbps	10 Mbps
Latency	10 ms	10 ms	1 ms

driving. The available total bandwidth of the network is 10MHz, and the minimum allocation unit is 200kHz. The resource allocation algorithm for network slicing makes decisions (adjustments) every 1,000ms. The total number of users is 1,200, who move in different settings. The environment size is 240m * 240m. It is assumed that users of the same type have the same moving direction and speed, and change direction by reflection when encountering the system boundary. The user mobility patterns in the table are set as default. In the experiment, the number of user devices can be increased, and the user movement speed can be set to fluctuate with certain distributions. The size distribution of data packets and SLA are set according to 3GPP TR 36.814 [34] and TS 22.261 [35]. The wireless base station covers a radius of 40m. The system aims to maximize the utility function of the base station. Table 2 shows the configuration parameters of the simulation environment.

B. EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

The weight α of the objective function is set to 0.01 by default, and the SSR weight vector β for each type of slicing service is set to [1,1,1] to indicate equal importance.

$$r = \begin{cases} 4 + (SE - 280) * 0.1, & q_v \geq 0.98 \text{ and } q_e \geq 0.98 \text{ and } q_u \geq 0.95 \text{ and } SE \geq 280 \\ 4, & q_v \geq 0.98 \text{ and } q_e \geq 0.98 \text{ and } q_u \geq 0.95 \text{ and } SE < 280 \\ (q_u - 0.7) * 10, & q_v \geq 0.98 \text{ and } q_e \geq 0.98 \text{ and } q_u < 0.95 \\ -5, & q_v < 0.98 \text{ or } q_e < 0.98 \end{cases} \quad (15)$$

As there is a gap between the magnitudes of SE (usually above 200bps/Hz) and SSR, α is set small to scale SE to the same level as SSR. Otherwise, the agent would essentially optimize only for SE, as the impact of SSR on the objective function is negligible. In the comparative experiments, the following algorithms are used for performance comparison:

- 1) EVEN: Distributing total bandwidth evenly to each slice;
- 2) DQN [25]: a bandwidth allocation policy generated by training a model using the DQN reinforcement learning algorithm for 10,000 rounds;
- 3) A2C [7]: A bandwidth allocation policy generated by training a model using the A2C reinforcement learning algorithm for 10,000 rounds;
- 4) A2C with LSTM (the algorithm designed in this section): A A2C reinforcement learning method that uses two queues to store environmental states, processes them through LSTM networks and concatenates the state features.

Based on the simulation environment settings in Section VI and the training process of Algo. 1, Fig. 4 shows a comparison of the reward feedback obtained by different algorithms after 10,000 rounds of training. Overall, reinforcement learning algorithms can achieve higher reward feedback through sufficient training. In the latter half of the training process when the algorithm model tends to converge, the A2C algorithm with LSTM achieves the highest average reward, followed by the A2C algorithm and DQN. According to the reward trend of each algorithm in the figure, the algorithm that preprocesses the state by passing two long- and short-term state queues through an LSTM network can achieve higher rewards during training than the A2C algorithm. After 6,000 rounds of training, the A2C algorithm with LSTM can achieve reward values of 8-10, while the A2C algorithm can only achieve reward values lower than around 8. As this implies, the mobility characteristics of users do affect the environmental state, and the LSTM network can indeed extract features of service request states time series. The large fluctuation range of the reward obtained by the DQN algorithm is due to its learning process, which involves sampling experiences from the experience pool for updates. In bandwidth resource allocation decisions, slight differences in bandwidth allocation can lead to significant fluctuations in the SSR of a network slice. These poor bandwidth allocation policies are stored as experience in the memory pool, affecting the stability of policy. Plus, the DQN algorithm has a slow convergence rate. The A2C algorithm achieves the fastest convergence rate, and after 1,000 rounds of training, its reward remains at around 7.8. The convergence rate of the A2C algorithm with LSTM is slower than that of the A2C algorithm because its input includes features of past state sequences. Although unstable state sequence features delay the convergence rate of the algorithm, they help make better bandwidth resource allocation decisions to obtain higher rewards. In addition, the A2C algorithm with LSTM introduces an entropy term in the loss function of the actor network to encourage policy

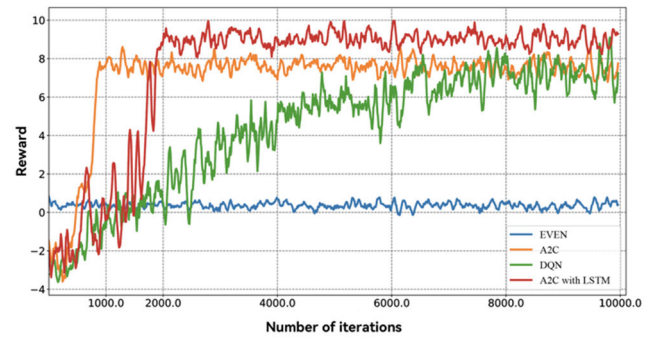


FIGURE 4. Comparison of algorithm rewards.

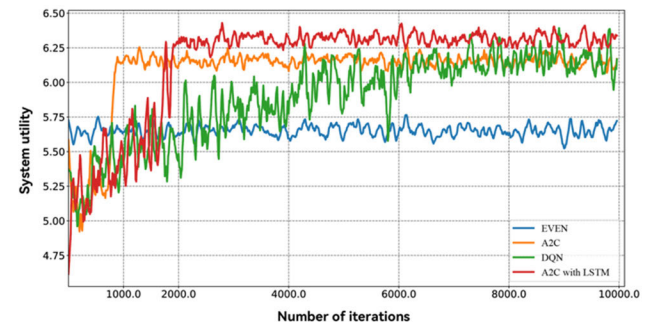


FIGURE 5. Comparison of system utility performance.

exploration in the action space. The EVEN allocation policy maintains the same bandwidth resources for each slice, rather than dynamically allocating bandwidth resources according to their changing service requirements, which explains its low reward values and a small fluctuation range.

Fig. 5 shows the system utility achieved by different algorithms during a training period of 10,000 rounds. The system utility function is defined in Section IV, and the optimization objective of this section is the system utility indicator, which consists of SE and SSR of each slice service. System utility can comprehensively reflect the overall performance of an algorithm. In general, the trend of changes in the system utility of algorithms is the same as the performance trend in the reward function experiment, indicating that the reward function set in Section V can guide the model to optimize the system utility objective. As shown in the figure, the A2C algorithm with the LSTM network still achieves the best system utility. The DQN algorithm achieves the second-best performance in the later stage of training, but the system utility value fluctuates greatly and converges slowly. After rapid trial and error, the A2C algorithm enters the convergence state after 1,000 rounds, but the achieved system utility lags behind that of the A2C network with LSTM. The experimental results show that user mobility has an impact on their service requirements and has certain temporal characteristics. Adding the LSTM network to the processing of state sequences can effectively help the algorithm improve the system utility in a real and valid way.

Fig. 6 presents a comparison of SEs for different algorithms. Considering the preciousness of wireless bandwidth

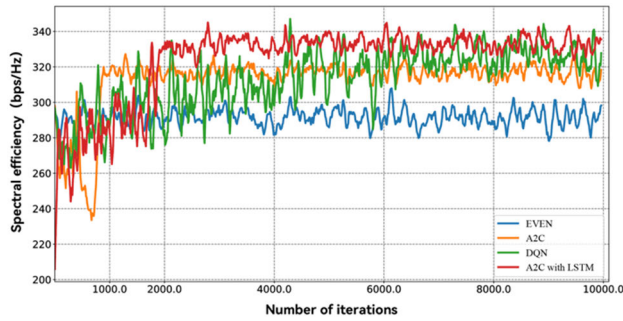


FIGURE 6. Comparison of SE.

resources, improving SE can not only provide higher transmission rates for users but also save wireless spectrum resources. As shown in the figure, the A2C algorithm with LSTM tends to converge at 2,000 rounds, while SE remains around 330. The DQN algorithm achieves the second-best performance but has significant fluctuations. In the subsequent bandwidth resource allocation diagram of the DQN algorithm, the reasons for the large range of fluctuations in the DQN algorithm will be analyzed from the perspective of allocation policies. According to the calculation formula of SE, even if the same total bandwidth is allocated to all slices, due to the impact of user mobility on the signal-to-noise ratio, although the algorithm's calculation model for the signal-to-noise ratio is unknown, a model that takes into account the impact of demand changes due to user mobility is obtained through training and inputting the state sequence into the LSTM network, and the SE performance indicator can be optimized accordingly.

Fig. 7 depicts a comparison of SSRs for VoLTE service. As shown in the figure, almost all algorithms can meet the requirements of this service. The SSR of VoLTE service for reinforcement learning algorithms has slight fluctuations, with a small amount of packet loss and transmission failures, but these can be almost ignored. The EVEN allocation policy maintains an SSR of 1.0, as one-third of the total bandwidth resources are sufficient to meet the requirements of VoLTE service. In fact, according to the VoLTE settings in Section VI, VoLTE service has low requirements and its SSR can be met by a small amount of bandwidth allocation. Subsequent experimental results show that only the minimum bandwidth allocation unit (0.2 MHz) is required to ensure the SSR of this service.

Fig. 8 shows the SSRs of different algorithms for eMBB service slicing. Overall, all reinforcement learning algorithms can improve the SSR of eMBB service slicing after training, but the DQN algorithm fails to guarantee a 0.98 SSR threshold even after 10,000 rounds of training. Both the EVEN allocation policy and two A2C-based algorithms can ensure the SSR of this service slicing is above 0.98.

Fig. 9 shows a comparison of SSRs for URLLC service. Overall, the A2C algorithm and A2C algorithm with LSTM can maintain an SSR of around 0.98 after training, while the DQN algorithm still widely exhibits SSR below 0.9 after

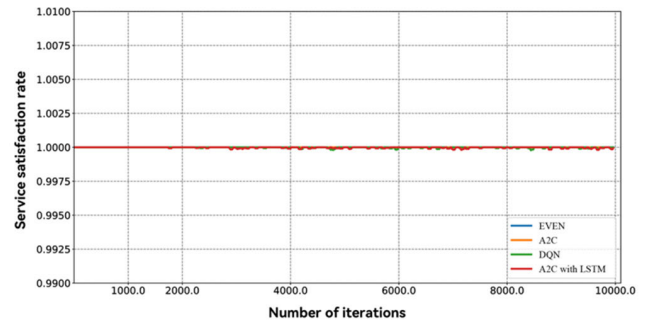


FIGURE 7. Comparison of SSRs for VoLTE service.

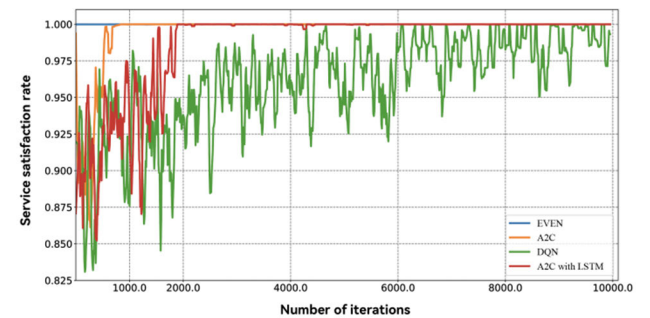


FIGURE 8. Comparison of SSRs for eMBB Service.

8,000-10,000 rounds of training, which obviously cannot meet the demand of guaranteeing the multiservice SSR in network slicing. URLLC is the typical request with high requirements for latency and reliability. In the experiment, the weight vector β for different services is set as [1,1,1], but there remain some services that cannot be satisfied. In subsequent experiments, the weight factor for URLLC service can be increased to contribute more to system utility and ensure that all URLLC service requests are met. In the Even allocation policy, although the URLLC service slice is allocated one-third of the total bandwidth, it only achieves an SSR between 0.7 and 0.8. Obviously, more bandwidth resources should be allocated to the URLLC service slice. Although URLLC service requires a lower transmission rate, they require extremely high latency, so the packets must be transmitted quickly to avoid packet loss due to exceeding the response latency.

To investigate the reasons for the performance differences among the algorithms, the experiment will train A2C, DQN, and A2C algorithms with LSTM for 20,000 rounds, and analyze their performance differences by observing their bandwidth allocation policies (i.e., actions in each round). The simulation environment and training configuration are kept default, and the maximum number of iterations is set to 20,000 rounds. Fig. 10 shows the bandwidth allocation of the A2C algorithm during the 20,000-round training process. It can be seen that the bandwidth allocation policy of the A2C algorithm fluctuates over a large range in the first 1,000 rounds of training. After 1,000 rounds, the model begins to

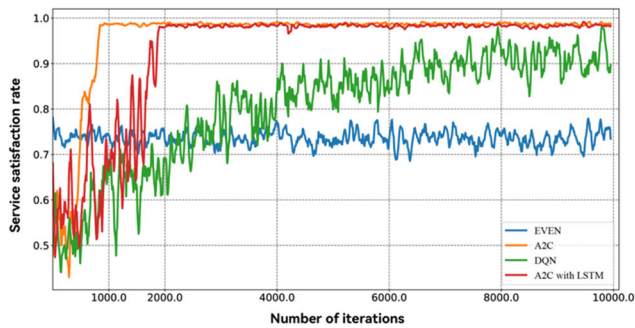


FIGURE 9. Comparison of SSRs for URLLC service.

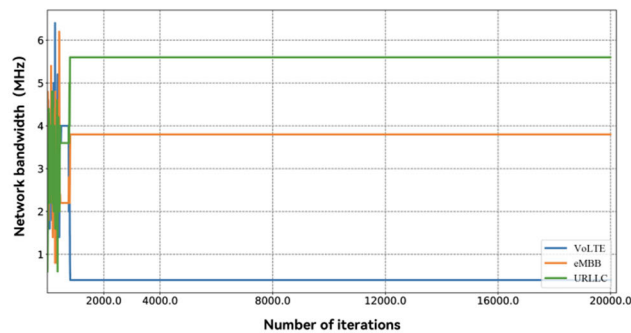


FIGURE 10. Bandwidth allocation of A2C algorithm.

converge. VoLTE service maintains a bandwidth allocation of 0.4 MHz, eMBB service 3.8MHz, and URLLC service 5.6MHz. According to the comparison with Fig. 12, the A2C algorithm and the A2C algorithm with LSTM stabilize the bandwidth allocation policy at the same level after convergence. Therefore, the A2C algorithm can obtain more stable system utility than the DQN algorithm after convergence. As shown in Fig. 6, the A2C algorithm with LSTM obtains higher SE because it allocates 4.2 MHz for the eMBB service slice, while A2C allocates only 3.8 MHz. The eMBB service requires a higher transmission rate, which helps to achieve a higher SE. Therefore, allocating more bandwidth to eMBB leads to differences in SE between the A2C algorithm with LSTM and the A2C algorithm. Furthermore, as shown in Fig. 9, allocating more bandwidth for URLLC service does not improve its SSR, as the service's un-SSR is more triggered by the higher packet loss rate caused by its high latency requirement. In fact, allocating only 0.2 MHz of bandwidth to VoLTE service is sufficient to meet its SSR, and allocating more bandwidth resources does not improve its SE.

Fig. 11 shows the bandwidth resource allocation by the DQN algorithm for three network slicing services during 20,000 rounds of training. In the first 8,000 rounds of training, the bandwidth allocation policy of the DQN algorithm fluctuates significantly, resulting in oscillations in the rewards it obtains. Also, this verifies that the rewards, system utility, and SE achieved by the DQN algorithm in Figs 4, 5, and 6 all fluctuate in a wide range. This is because as a value-based reinforcement learning algorithm, DQN judges the value of

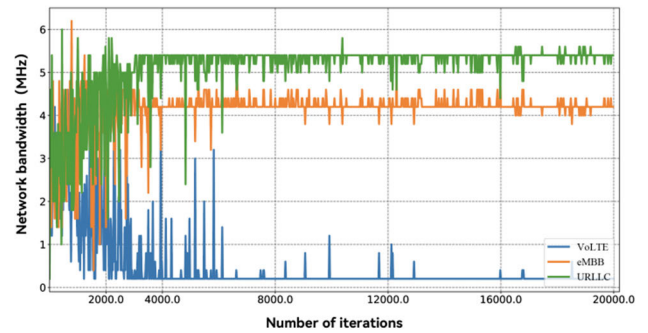


FIGURE 11. Bandwidth allocation of DQN algorithm.

actions based on the Q-value, which undermines the stabilization of the policy in a particular direction. Moreover, because the learning of the DQN algorithm at each step is based on the batch learning of experience in the memory pool, its convergence is slow, as it becomes more difficult to optimize the model towards a “better” direction due to previous poor feedback experiences in the batch. After 12,000 rounds of training, the bandwidth allocation of the DQN algorithm can be stabilized within a small range. Compared to Fig. 12, the DQN algorithm allocates about 0.2 MHz of bandwidth to the network slice of VoLTE service, 4.2 MHz to the slice of eMBB service, and 5.4 MHz to the slice of URLLC service. However, there are still fluctuations in a certain range, so the SE performance of the DQN algorithm is comparable to that of the A2C algorithm with the LSTM network. Due to the volatility of the bandwidth allocation policy, the SSR of network slicing services cannot be guaranteed, as shown in Figs 8 and 9. From above, because of the instability of the DQN algorithm's decision-making, it cannot ensure the SSR of network slicing services, resulting in the performance gap between the DQN algorithm and the A2C algorithm with LSTM in terms of system utility.

Fig. 12 shows the bandwidth allocation of the A2C algorithm with the LSTM network during 20,000 rounds of training. In general, the bandwidth allocation policy of this algorithm tends to be stable after 2,000 rounds of training, consistent with the rewards, system utility, and SE performance of the algorithm shown in Figs 4, 5, and 6 after 2,000 rounds of training. The bandwidth resource allocation for the VoLTE service's network slice eventually stabilizes at 0.2 MHz, because the service's data packet size is constant, the user reach interval is uniformly distributed, and the service requires a low transmission rate and insensitive latency (as shown in Section VI simulation environment settings). The bandwidth allocation for the eMBB service's network slice eventually stabilizes at 4.2 MHz, because the service requires a higher transmission rate of data packets and needs more bandwidth allocation. The URLLC service has larger data packets and more users in the simulation settings and requires high latency. Therefore, in the experimental configuration, the algorithm allocates the most bandwidth resources, which is 5.4 MHz, to the network slice of URLLC

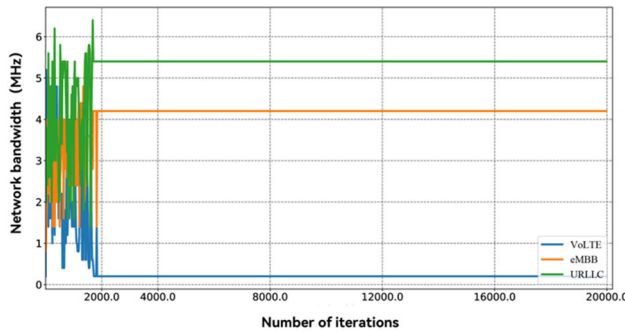


FIGURE 12. Bandwidth allocation of A2C algorithm with LSTM network.

service. It can be seen that in the simulation environment settings, the total bandwidth resource of the wireless base station is 10 MHz, while the total allocated resources for the three slices is 9.8 MHz. This is because the bandwidth allocated to them can already meet their service quality requirements. According to the SE formula, the larger the allocated bandwidth, the larger the denominator of the formula and the lower the SE, as long as the service rate requirement is met. Therefore, the algorithm will not allocate an extra 0.2 MHz bandwidth to other slices to improve the SE, i.e. the utilization of wireless bandwidth resources. In the simulation environment settings and system model, when users move, their positions change, resulting in changes in the data demands of each slice service. However, after 2,000 rounds of training, the algorithm maintains the same bandwidth allocation policy. The reasons for this include: first, after 2,000 rounds of training, the algorithm learns a policy that can largely obtain better reward feedback, which means the algorithm has learned how the changes in users affect the changes in service demands in the environment. Moreover, this is considered that even if the users move and cause changes in service demands, maintaining this allocation scheme can ensure the SSR of each slice service and obtain a higher SE. In fact, according to the SSR threshold set in the reward function, it is only necessary to ensure that q_v , q_e , and q_u are higher than 0.98, 0.98, and 0.95, respectively. Second, during environmental changes, two queues are used to store the state sequence and preprocess them with an LSTM network to obtain temporal features. When the past policy becomes stable, the environmental state sequence also tends to be stable under that policy, making it easier to make the same decisions in case of a similar length of the past state sequence.

Through comparison of the experimental results, it can be concluded that the proposed solution in this section can support various types of services with different needs in network slicing and guarantee their SSR. Meanwhile, to address the changing service demand caused by user mobility in the MEC scenario, the solution adopts an LSTM network, which can extract temporal features to help the A2C algorithm achieve better performance. Plus, the SE of the wireless network can be maintained at a relatively high level. All of these indicate that the

solution in this section can effectively solve the defined problem.

VII. CONCLUSION AND FUTURE WORK

Existing research on wireless network slicing has insufficiently accounted for the impact of users' mobility characteristics on their service requirements. This lack of consideration in dynamically allocating network slice resources to mobile users results in unfulfilled service demands. In the scenario of MEC, this paper systematically examines the bandwidth allocation policy for RAN network slicing, realizing wireless network service and a guarantee for user network services. Based on three typical service types, the study dynamically allocates wireless bandwidth resources to slices to meet different service requirements and improve the SE of wireless bandwidth. Besides, user mobility is taken into consideration for bandwidth allocation decision-making, so as to support changes in network slicing service requirements in the MEC scenario. A solution based on reinforcement learning and other advanced neural network modules is adopted to achieve specific feature extraction, which can effectively allocate limited wireless bandwidth resources and improve wireless SE while ensuring slice service SE.

As the system models in the study are all based on real scenarios, they have great practical significance. The optimization of system objectives is expected to improve user experience. The results of simulation experiments verify the effectiveness of the solution. Therefore, while applying it to real-world systems requires addressing practical engineering challenges such as data collection, algorithm implementation, and performance evaluation, the method proposed in this paper has demonstrated practical applicability in both theoretical optimization of wireless network slice resource allocation and experimental simulations.

Regarding network slicing in MEC, future research may adopt the following perspectives: first, introducing other technologies (e.g., blockchain-based technologies) to enhance network resource security; second, incorporating network resource allocation for different users within the slice to achieve intra-slice utility optimization.

REFERENCES

- [1] S. Ahmadi, *5G NR: Architecture, Technology, Implementation, and Operation of 3GPP New Radio Standards*. New York, NY, USA: Academic, 2019.
- [2] A. Banchs, G. de Veciana, V. Sciancalepore, and X. Costa-Perez, "Resource allocation for network slicing in mobile networks," *IEEE Access*, vol. 8, pp. 214696–214706, 2020, doi: 10.1109/ACCESS.2020.3040949.
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [4] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
- [5] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1, pp. 1–7.

- [6] R. Y. Li, H. M. Peng, R. G. Li, and K. Zhao, "Overview on algorithms and applications for reinforcement learning," *Comput. Syst. Appl.*, vol. 29, no. 12, pp. 13–25, 2020.
- [7] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [8] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [10] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [11] J. Khamse-Ashari, G. Senarath, I. Bor-Yaliniz, and H. Yanikomeroglu, "An agile and distributed mechanism for inter-domain network slicing in next generation mobile networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3486–3501, Oct. 2022.
- [12] A. Lieto, I. Malanchini, S. Mandelli, E. Moro, and A. Capone, "Strategic network slicing management in radio access networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 4, pp. 1434–1448, Apr. 2022.
- [13] A. Thantharath, R. Paropkari, V. Walunj, and C. Beard, "DeepSlice: A deep learning approach towards an efficient and reliable network slicing in 5G networks," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, Oct. 2019, pp. 762–767.
- [14] L. Zanzi, A. Albanese, V. Sciancalepore, and X. Costa-Pérez, "NSBchain: A secure blockchain framework for network slicing brokerage," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–7.
- [15] M. A. Togou, T. Bi, K. Dev, K. McDonnell, A. Milenovic, H. Tewari, and G.-M. Muntean, "DBNS: A distributed blockchain-enabled network slicing framework for 5G networks," *IEEE Commun. Mag.*, vol. 58, no. 11, pp. 90–96, Nov. 2020.
- [16] I. H. Abdulqader and S. Zhou, "SliceBlock: Context-aware authentication handover and secure network slicing using DAG-blockchain in edge-assisted SDN/NFV-6G environment," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 18079–18097, Sep. 2022.
- [17] Y. Azimi, S. Yousefi, H. Kalbkhani, and T. Kunz, "Applications of machine learning in resource management for RAN-slicing in 5G and beyond networks: A survey," *IEEE Access*, vol. 10, pp. 106581–106612, 2022.
- [18] M. Setayesh, S. Bahrami, and V. W. S. Wong, "Resource slicing for eMBB and URLLC services in radio access network using hierarchical deep learning," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 8950–8966, Nov. 2022.
- [19] Z. Wang, Y. Wei, F. R. Yu, and Z. Han, "Utility optimization for resource allocation in multi-access edge network slicing: A twin-actor deep deterministic policy gradient approach," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 5842–5856, Aug. 2022.
- [20] K. Boutiba, M. Bagaa, and A. Ksentini, "Radio resource management in multi-numerology 5G new radio featuring network slicing," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2022, pp. 359–364.
- [21] K. Boutiba, A. Ksentini, B. Brik, Y. Challal, and A. Balla, "NRflex: Enforcing network slicing in 5G new radio," *Comput. Commun.*, vol. 181, pp. 284–292, Jan. 2022.
- [22] T. Mai, H. Yao, N. Zhang, W. He, D. Guo, and M. Guizani, "Transfer reinforcement learning aided distributed network slicing optimization in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 4308–4316, Jun. 2022.
- [23] S. Messaoud, A. Bradai, O. B. Ahmed, P. T. A. Quang, M. Atri, and M. S. Hossain, "Deep federated Q-learning-based network slicing for industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5572–5582, Aug. 2021.
- [24] Y. Xu, Z. Zhao, P. Cheng, Z. Chen, M. Ding, B. Vucetic, and Y. Li, "Constrained reinforcement learning for resource allocation in network slicing," *IEEE Commun. Lett.*, vol. 25, no. 5, pp. 1554–1558, May 2021.
- [25] R. Li, Z. Zhao, Q. Sun, I. Chih-Lin, C. Yang, X. Chen, M. Zhao, and H. Zhang, "Deep reinforcement learning for resource management in network slicing," *IEEE Access*, vol. 6, pp. 74429–74441, 2018.
- [26] Y. Shao, R. Li, B. Hu, Y. Wu, Z. Zhao, and H. Zhang, "Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 10792–10803, Oct. 2021.
- [27] Y. Hua, R. Li, Z. Zhao, X. Chen, and H. Zhang, "GAN-powered deep distributional reinforcement learning for resource management in network slicing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 334–349, Feb. 2020.
- [28] Y. Abiko, T. Saito, D. Ikeda, K. Ohta, T. Mizuno, and H. Mineno, "Flexible resource block allocation to multiple slices for radio access network slicing using deep reinforcement learning," *IEEE Access*, vol. 8, pp. 68183–68198, 2020.
- [29] Y. Kim and H. Lim, "Multi-agent reinforcement learning-based resource management for end-to-end network slicing," *IEEE Access*, vol. 9, pp. 56178–56190, 2021.
- [30] Y. Cui, X. Huang, P. He, D. Wu, and R. Wang, "QoS guaranteed network slicing orchestration for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 15215–15227, Aug. 2022.
- [31] R. Li, C. Wang, Z. Zhao, R. Guo, and H. Zhang, "The LSTM-based advantage actor-critic learning for resource management in network slicing with user mobility," *IEEE Commun. Lett.*, vol. 24, no. 9, pp. 2005–2009, Sep. 2020.
- [32] Q. Liu, T. Han, N. Zhang, and Y. Wang, "DeepSlicing: Deep reinforcement learning assisted resource allocation for network slicing," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2020, pp. 1–6.
- [33] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [34] *Evolved Universal Terrestrial Radio Access (E-UTRA); Further Advancements for E-UTRA Physical Layer Aspect*, document TR 36.814, 3GPP, 2015.
- [35] *Service Requirements for the 5G System*, document TS 22.261, 3GPP, 2017.



XIAOLEI CHANG was born in China, in 1975. He received the bachelor's degree in computer science and technology, the master's degree in management science and engineering, and the Ph.D. degree in engineering (electronic information) from Tsinghua University, in 1999 and 2001, respectively.

He is currently a Senior Engineer with the Information Technology Center/Network Research Institute, Tsinghua University. He is also the Deputy Director of the Next Generation Internet Research and Development Center, Research Institute of Tsinghua University in Shenzhen. He has been engaged in information technology application research and engineering practice, transformation of scientific and technological achievements, and technology enterprise incubation for a long time. He has completed more than 20 research projects. He has published more than 20 papers and two books. His research interests include green data center, cloud computing and edge computing, and cyber space security.



TIAN JI was born in Jingdezhen, China, in 1989. He received the M.S. degree in big data from Tsinghua University, in 2023. He is currently a core member of the Future Internet Research Center, Research Institute of Tsinghua University in Shenzhen. His research interests include the application of big data, cloud computing, and edge computing technologies in areas, such as smart cities and the industrial internet. He has participated in digital transformation projects for multi-

ple local governments and leading state-owned enterprises, covering various aspects, including but not limited to data management and analysis, system integration, network architecture, and application development.



RUNSU ZHU was born in Guangdong, China, in 1981. He received the Graduate degree with a major in computer science and technology from Sun Yat-sen University, in 2003, and the master's degree from Xiamen University, in 2010.

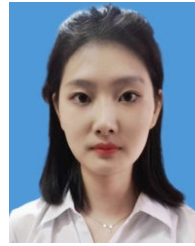
He has been with Shenzhen's Civil Service System, since May 2004. He is currently the Party Secretary and the Director of Shenzhen Longhua District Government Service Data Administration. He has published two monographs, three papers,

undertook eight projects, and participated in the formulation of one national standard. He has a professional title of senior engineer. His research interest includes electronic information field. He can make innovative scientific research results and effectively promote the transformation of scientific research results in the industry.



ZHENZHOU WU received the bachelor's degree in software engineering from Jilin University, in 2009. From 2009 to 2011, he was a Search Engineer with AvePoint, responsible for the company's full-text search department. From 2011 to 2014, he was the Background Architect with Fenglin Volcano Technology Company Ltd., supporting the game platform with tens of millions of users and millions of users online at the same time. From 2014 to 2017, he was the Research and

Development Director of Oudmon Technology Company Ltd., to develop the IoT platform, which has access to more than 50 million wearable devices. He is currently a senior architect, a lisp programmer, and the research and development director of several internet companies. Since 2017, he has been the Research and Development Director of the Next Generation Internet Research and Development Center, Research Institute of Tsinghua University in Shenzhen. His research interests include cloud native, distributed networks, and distributed computing.



CHENXI LI was born in Chenzhou, Hunan, China, in 1999. She received the bachelor's degree in materials science and engineering from the China University of Mining and Technology, in 2020. Since 2021, she has been a Research Assistant with the Next Generation Internet Research and Development Center, Research Institute of Tsinghua University in Shenzhen. Her research interests include the development and application of 5G, edge computing, and other new generation information technologies.



YONG JIANG received the M.S. and Ph.D. degrees in computer science from Tsinghua University, Guangdong, China, in 1998 and 2002, respectively. Since 2002, he has been with the Tsinghua Shenzhen International Graduate School, Tsinghua University, where he is currently a Full Professor. His research interests include computer vision, machine learning, internet architecture and its protocols, and IP routing technology. His research has published in multiple top-tier journals and conferences, including IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON MULTIMEDIA, IEEE TRANSACTIONS ON SIGNAL PROCESSING, CVPR, and ICLR. He has received several best paper awards (e.g., IWQoS 2018) from top-tier conferences.